Théorie des langages

2ème Année Informatique Appliquée

Théorie des langages

Plan

- Alphabets et langages
- Expressions régulières formelles

Théorie des langages

Alphabets et langages

- Structure du langage :
  - Algèbre, **ensemble** muni d'**opérations** (anneau)
  - « Briques de base » : **alphabet**
  - **Propriétés formelles** particulières du langage
- Différentes manières de définir un langage :
  - Intentionnelle : « tous les mots qui... »
  - Extensionnelle : {mot, autremot, a, b, 3, 42 }
  - Définitoire : {xyz | x = yz }
- **Opération sur d'autres langages** (union, concaténation, intersection, complémentaire ...)

Théorie des langages

Alphabets et langages

- **Alphabet**, un ensemble « hors-langage » :
  - Ensemble **∑** des symboles utilisés par un langage
    - ∑ = {a, b, c, d … z} (= [a-z])
    - ∑ = {0, 1, 2... 9} (= [0-9])
    - ∑ = {a, f, d, x}
    - ∑ = {b, c, 0, $, µ, z}
    - ∑ = {ab, de, xyz}
    - ∑ = {abc, a, tuv, vu}
  - On considère les éléments comme **atomiques** : « abc » peut- être un symbole unique pour un langage
  - Pas de répétitions au sein d'un alphabet (ensemble)

Théorie des langages

Alphabets et langages

- **Mot** (ou **chaîne**) :
  - Un élément de l'alphabet est un mot
  - **Concaténation « . » de symboles** issus d'un alphabet :
    - $\Sigma = \{0, 1\}$: $\alpha_1 = 0.0.1.0$, $\alpha_2 = 1.0.0.0.1.0$ ...
    - Associative : (a.b).c = a.(b.c)
    - Non commutative : $0.1 \neq 1.0$
  - Quelque soit l'alphabet, il est possible de former le **mot** (ou chaîne) **vide**, noté « **ε** » (différent de ∅)
  - **Taille du mot** : l x l, nombre de **symboles** :
    - l 0.1.0.0 l = 4 ($\Sigma = \{0, 1\}$)
    - l d.d.ab.ab.d l = 5 ($\Sigma = \{ab, d\}$)
    - l ε l = 0 ($\forall \Sigma$)

Théorie des langages

Alphabets et langages

- Un langage est un **ensemble de mots**
- Définition de langages à partir d'**alphabets** :
  - Langage généré par un alphabet (récursive) :
    - Tout mot de l'alphabet appartient au langage
    - Toute concaténation de mots du langage appartient au langage
    - Le mot vide ε appartient au langage
  - **Puissance** « Σ^i » d'un alphabet :
    - Tous les mots formés par concaténation de i éléments de Σ
    - Quelque soit l'alphabet, $\Sigma^0 = \{ \varepsilon \}$
    - Par ex., si $\Sigma = \{0, 1\}$ alors :
      - $\Sigma^1 = \{0, 1\}$
      - $\Sigma^2 = \{00, 01, 10, 11\}$
      - $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

Théorie des langages

Alphabets et langages

- **Fermeture / étoile de Kleene « * »** :
  - Kleene : mathématicien américain
  - Fermeture (étoile, itéré) (informellement) : tout ce qu'il est possible de construire à partir d'un alphabet / langage
  - Union de **toutes les puissances** d'un alphabet :
    - $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \Sigma^4 \ldots$
  - Ou « **langage de Σ** », « **langage généré par Σ** »
  - Quelque soit Σ : $\varepsilon \in \Sigma^*$
- Attention à la distinction **symboles / mots** :
  - Si $\Sigma = \{0, 11\}$ alors 1.0.1 n'appartient pas à Σ^*
  - Si $\Sigma = \{a, ab\}$ alors a.ab.b.ab. n'appartient pas à Σ^*

Théorie des langages

Alphabets et langages

- **Langage sur un alphabet** :
  - Un langage L sur un alphabet Σ est un sous-ensemble de Σ^*
- **Opérations** sur les langages (étant donné Σ) :
  - **Concaténation** : « L.M » (ou « LM ») : mot formés par concaténation d'éléments de L et de M (dans l'ordre)

    Associative, non commutative, élément neutre ε, élément absorbant ø
  - **Puissance** : « L^i » = { $\alpha \in L^*$ t. q. $|\alpha| = i$ } (idem alphabet)
    - Remarque : $L^0 = \varepsilon$ et $L^i = L^{i-1}.L$
  - **Fermeture** (étoile, itéré) : « L^* » = $\bigcup_{i \geq 0} L^i$ (idem alphabet)
    - Idempotente : $(L^*)^* = L^*$
    - Quelque soient L et Σ : $\varepsilon \in L^*$ (idem alphabet)

Théorie des langages

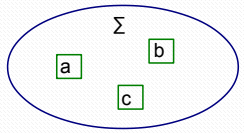
Alphabets et langages

- **Union** : « $L \cup M$ » (ou $L + M$) mots issus soit de L soit de M
  - Associative, commutative, élément neutre ø
  - La concaténation est distributive par rapport à l'union
- **Intersection** : « $L \cap M$ » : mots qui existent dans L et dans M
  - Associative, commutative, élément neutre Σ^*, élément absorbant ø
- **Complémentaire** : « $\overline{L}$ » : mots de Σ^* qui ne sont pas dans L
- **Fermeture** (étoile, itéré) **stricte** : : « L^+ » = $\bigcup_{i \geq 1} L^i$
  - Remarques : idempotente, $L^+ = L.L^*$ et $L^* = L^+ \cup \{\varepsilon\}$

Théorie des langages

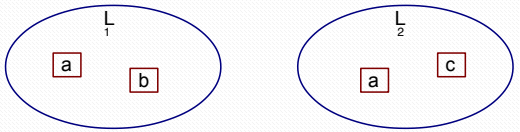
Alphabets et langages

- Quelques exemples :
  - Alphabet $\Sigma = \{ a, b, c \}$



Éléments de l'alphabet

  - Langages $L_1 = \{ a, b \}$, $L_2 = \{ a, c \}$

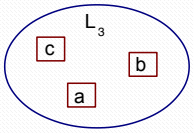


Éléments du langage

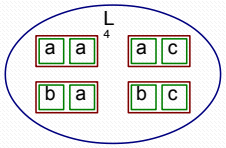
Théorie des langages

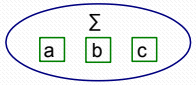
Alphabets et langages

- Quelques exemples (suite) :
  - $\Sigma = \{ a, b, c \}$, $L_1 = \{ a, b \}$, $L_2 = \{ a, c \}$
  - Union : $L_3 = L_1 \cup L_2$

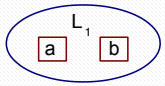


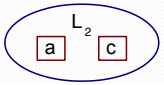
  - Concaténation : $L_4 = L_1 . L_2$





Éléments de l'alphabet



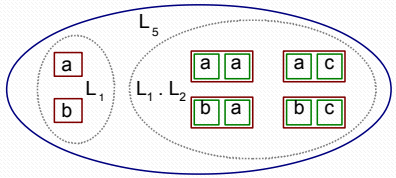


Éléments du langage

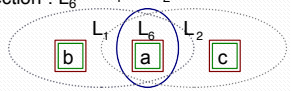
Théorie des langages

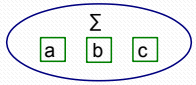
Alphabets et langages

- Quelques exemples (suite) :
  - $\Sigma = \{ a, b, c \}$, $L_1 = \{ a, b \}$, $L_2 = \{ a, c \}$
  - Union et concaténation : $L_5 = L_1 \cup (L_1 . L_2)$

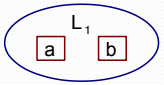


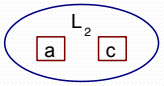
  - Intersection : $L_6 = L_1 \cap L_2$





Éléments de l'alphabet

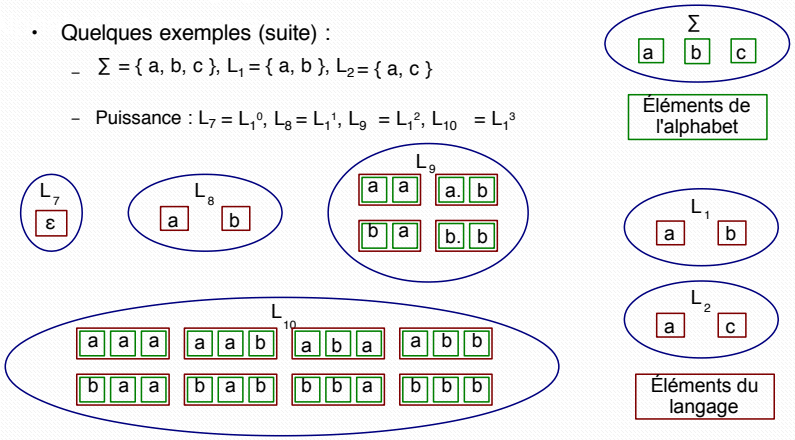




Éléments du langage

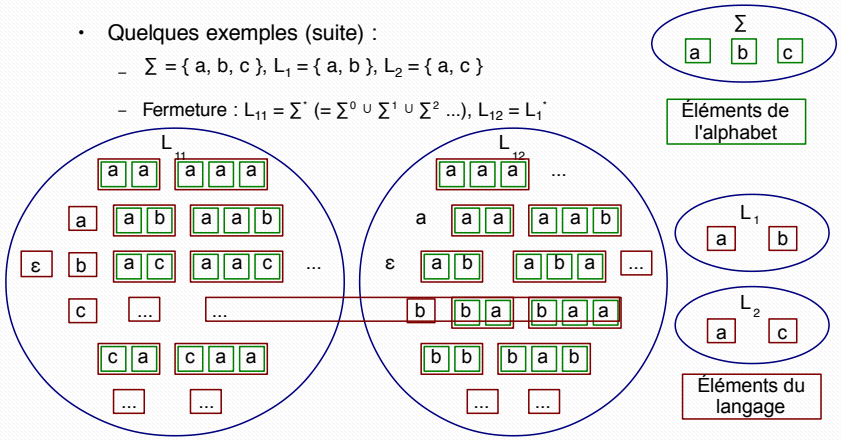
Alphabets et langages

- Quelques exemples (suite) :
  - $\Sigma = \{ a, b, c \}$, $L_1 = \{ a, b \}$, $L_2 = \{ a, c \}$
  - Puissance : $L_7 = L_1^0$, $L_8 = L_1^1$, $L_9 = L_1^2$, $L_{10} = L_1^3$



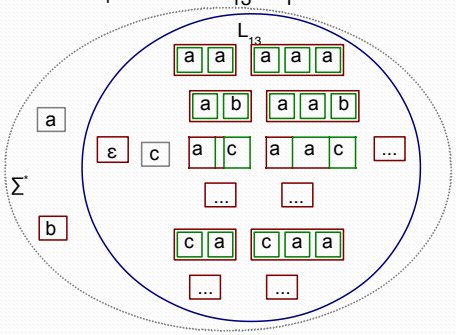
Alphabets et langages

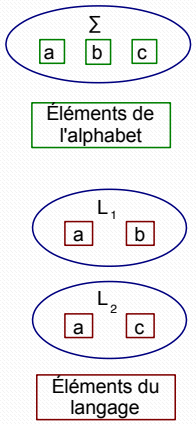
- Quelques exemples (suite) :
  - $\Sigma = \{ a, b, c \}$, $L_1 = \{ a, b \}$, $L_2 = \{ a, c \}$
  - Fermeture : $L_{11} = \Sigma^*$ ($= \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \ldots$), $L_{12} = L_1^*$



Alphabets et langages

- Quelques exemples (suite) :
  - $\Sigma = \{ a, b, c \}$, $L_1 = \{ a, b \}$, $L_2 = \{ a, c \}$
  - Complémentaire : $L_{13} = \overline{L_1}$ (≈ «$\Sigma^* - L_1$»)





Plan

- Alphabets et langages
- Expressions régulières formelles

Théorie des langages

Expressions régulières formelles

- **Expressions régulières** :
  - ≃ expressions **rationnelles** (langage régulier ≃ rationnel)
  - Formalisées mathématiquement en 1959 (Rabin & Scott)
  - Utilisées en programmation (regexp) : **grep**, awk, Perl, Python
- Un « **langage pour définir un langage** »
  - Symboles (par priorité) : { **(,), *, $^+$, +**}
  - Une expression régulière « génère », « **accepte** », « **reconnaît** » un langage (dit régulier)
- Langage des expression régulières sur ∑ : **Reg(∑)**
  - Reg(∑) ⊂ (∑ ∪ { **+**, *, $^+$, **(,)** })*
- Programmation : le + deviendra | (éviter la confusion)

Théorie des langages

Expressions régulières formelles

- Définition **récursive** (étant donné ∑) :
  - Tout élément de ∑ est une expression régulière
  - Si r est une expression régulière, alors (r), r^+, r^* aussi
  - Si r_1 et r_2 sont des exp. régulières, alors $r_1 r_2$ et $r_1 + r_2$ aussi
- Soit l'**application L** qui associe à une **expression régulière** un **langage**, définie de la manière suivante :
  - L : Reg(∑) → Σ^*
  - L(a) = { a } ∀ a ∈ ∑ , L(ε) = { ε } et L(∅) = ∅
  - **$L(r_1 + r_2) = L(r_1) \cup L(r_2)$**
  - **$L(r_1 r_2) = L(r_1) . L(r_2)$**
  - **$L(r^*) = L(r)^*$ et $L(r^+) = L(r)^+$**

Théorie des langages

Expressions régulières formelles

- Par ex., soit ∑ = { a, b, c } :
  exercice d'application:
  - L(b) = b
  - L(a+c) = L(a) ∪ L(c) = {a, c}
  - L(ac) = L(a) . L(c) = {a.c}
  - $L(c^*) = L(c)^* = \{c\}^* = \{\varepsilon, c, cc, ccc, cccc...\}$
  - $L(a+c^*) = L(a) \cup L(c^*) = \{a\} \cup \{c\}^* = \{\varepsilon, a, c, cc, ccc, cccc...\}$
  - $L((a+b)^*) = (L(a) \cup L(b))^* = \{a, b\}^*$
    - = {ε, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab …}
  - $L((ac)^+ + b) = (L(a) . L(c))^+ \cup L(b) = \{ac\}^+ \cup \{b\}$
    - = {b, ac, acac, acacac …}
  - $L(a^+ + (abc)^* + ((b+a)c)^+) = L(a)^+ \cup L(abc))^* \cup L(\{b, a\}.c)^+$
    - = {a, aa, aaa … ε, abc, abcabc … bc, ac, bcbc, acac...}

Théorie des langages

Expressions régulières formelles

- Quelques cas d'applications classiques :
  - Trouver des fichiers dans un dossier :
    - Tous les fichiers d'extension « .jpg » : *.jpg
  - Chercher toutes les fonctions « get... » dans un programme :
    - Expression régulière : function get[a-Z]*(
  - Extraire toutes les phrases d'un texte :
    - Expression régulière : $\overline{(.+?+!)}^*(.+?+!)^*$
  - Vérifier si une entrée de programme est bien un entier :
    - Expression régulière : (-+ε){0...9)*
  - Chercher les mots au pluriel dans un texte :
    - Expression régulière : [a-z]*((e+a)ux+s)